Kutaisi International University

School of Computer Science

Bachelor's Educational Program in Computer Science

Erekle Bagrationi, Mari Kapanadze

Evaluating Large Language Models' Understanding of C0 Grammar Through Derivation Tree Generation

Thesis submitted for the academic degree of
Bachelor of Computer Science

Supervisor: **Date:**

Prof. Vasili Nikolaev June, 2026

Declaration of Authorship

We, Mari Kapanadze and Erekle Bagrationi, the authors of this bachelor's thesis, declare that the thesis is our own original work. To the best of our knowledge, it does not contain any material previously published. And if so, it is cited according to academic standards.

Mari Kapanadze, Erekle Bagrationi	06/18/2026
Student's Full Name	Date

Abstract

Large language models (LLMs) are becoming part of a programmers lifes. Even though LLMs are used so frequently, their abilities in formal grammar are still not well understood. Our thesis aims to investigate how well LLMs understand the grammar of the C0 programming language. We evaluated their abilities by asking them to generate derivation trees from given instructions.

We designed a controlled experiment for five different LLMs: ChatGPT (GPT-5.5), Gemini (3.5 Flash), Claude (Sonnet 4.6), Grok (4.6 Fast), and DeepSeek-V3. Each model was given the same instructions, which included: the complete C0 grammar, a strict output format, and 51 lines of code. code snippets included standard syntax, pointers, and intentionally out-of-grammar inputs. The results were compared to the original derivation trees generated by the C0 compiler. Comparison was done by using Selkow's tree edit distance and a grammar-validity. Both of which were used in the `tree_diff.html` file.

The results showed notable inconsistencies. Claude had highest mean structural similarity (0.864) and lowest mean edit distance (42.4). while, Gemini had the highest legality (0.946) for in-grammar programs, it stuggled with pointer tasks (0.284 in structural similarity). ChatGPT was the most consistant. It produced the most border-correct program derivations (23 out of 51).On the other hand, DeepSeek-V3 was the worst performing model. In addition,no model reliably rejected out-of-grammar inputs.these finding support the idea that LLMs do not internalize formal grammar rules independently.

Keywords: large language models, derivation trees, formal grammars, C0 programming language, tree edit distance, Selkow algorithm, syntax parsing, Mermaid diagrams, grammar evaluation

Table of Contents

List of Figures

List of Tables

1. Introduction

Background and Relevance

Compiler construction rests on formal language theory: a programming language is defined by a grammar, and a parser applies production rules to build a derivation tree (parse tree) that records how an input string was derived from the start symbol [3]. Unlike generated source code, a derivation tree exposes every intermediate syntactic decision. Correct tree construction therefore requires precise knowledge of non-terminal expansions, operator precedence, lexical decomposition, and the distinction between syntactic categories.

The C0 programming language evaluated in this thesis is a pedagogical dialect used in compiler courses at Kutaisi International University (KIU). It follows the tradition of teaching-oriented C subsets such as Carnegie Mellon's C0 [4, 5], but defines its own grammar with KIU-specific pointer notation (`'` for dereference, `&` for address-of, `*` for multiplication or pointer types). Constructing a derivation tree for a C0 program is a rigorous test of grammatical understanding that cannot be satisfied by approximate code generation alone.

Large language models (LLMs) have demonstrated strong performance on code benchmarks such as HumanEval [15] and MBPP [16], yet those benchmarks evaluate functional correctness of generated programs rather than step-by-step adherence to an explicit context-free grammar [12, 20]. Recent surveys confirm that most LLM-for-code research still models source as token sequences and prioritizes downstream software engineering tasks over formal parsing [12, 32]. As LLMs are adopted in education and compiler tooling, it becomes important to determine whether they understand grammar rules or merely reproduce statistically likely syntactic patterns [27]. This thesis addresses that question through a direct, automatable evaluation of derivation tree generation.

Research Questions

This study is guided by five research questions:

1. **RQ1:** Can modern large language models correctly generate derivation trees for valid C0 programs?
2. **RQ2:** Which large language model produces derivation trees most similar to those generated by the reference compiler?
3. **RQ3:** Can large language models correctly distinguish pointer syntax from multiplication expressions?

4. **RQ4:** How do large language models respond to syntactically invalid inputs that are outside the C0 grammar?
5. **RQ5:** Does high tree similarity indicate deeper understanding of formal grammar rules?

Research Aim and Objectives

The primary aim is to evaluate and compare the grammatical competence of selected LLMs in generating C0 derivation trees under controlled, identical conditions. Specific objectives are:

1. To define a reproducible evaluation framework (grammar, format standard, test corpus, comparison pipeline).
2. To collect derivation tree outputs from five LLMs for 51 test programs.
3. To compare each output against the reference compiler's trees using Selkow's tree edit distance and grammar-legality metrics.
4. To analyze performance across standard, pointer, and out-of-grammar categories.
5. To identify recurring error patterns and interpret results in terms of formal understanding versus pattern matching.

Research Subject and Scope

The research subject is syntactic derivation of C0 programs represented as directed Mermaid graphs. Scope boundaries:

- **Language:** C0 as defined in `instruction.txt`.
- **Output format:** Mermaid `graph TD` per the v2 format specification.
- **Models:** ChatGPT (GPT-5.5), Gemini 3.5 Flash, Claude Sonnet 4.6, Grok 4.6 Fast, DeepSeek-V3—one run per task, no few-shot examples.
- **Test corpus:** 44 in-grammar programs, 4 pointer programs, 3 out-of-grammar programs (`accepted C0 lines.txt`).
- **Ground truth:** Reference trees in `generated_trees/` from the reference C0 compiler.
- **Exclusions:** Semantic correctness, runtime behavior, and code optimization.

Practical Significance

Reliable derivation tree generation supports compiler education, automated syntax grading, and formal assessment of AI systems. The Mermaid-based pipeline developed here is human-readable, version-controllable, and reusable across models and grammar revisions. Conversely, systematic

failure modes—invented symbols, incomplete lexical expansion, pointer confusion, and forced derivation of invalid programs—reveal limits not visible in general coding benchmarks.

Thesis Structure

- **Chapter 1** introduces background, research questions, and scope.
- **Chapter 2** reviews formal language theory, derivation trees, LLMs, and related evaluation literature.
- **Chapter 3** describes methodology: dataset, prompts, models, and metrics.
- **Chapter 4** documents the implementation of tree comparison and result processing.
- **Chapter 5** presents quantitative results.
- **Chapter 6** discusses findings and answers the research questions.
- **Chapter 7** analyzes threats to validity.
- **Chapter 8** concludes and recommends future work.
- **Appendices** contain the grammar, format rules, test statements, and data locations.

2. Literature Review

This chapter situates the present study within formal language theory, compiler construction, tree comparison algorithms, and the rapidly growing literature on large language models for code. Rather than summarizing sources in isolation, the review compares methodological assumptions, highlights disagreements in the literature, and identifies the gap that derivation-tree evaluation addresses.

Formal Language Theory and Context-Free Grammars

A formal grammar $G = (V, \Sigma, P, S)$ consists of a finite set of non-terminal symbols V, terminal alphabet Σ, production set P, and start symbol S. A context-free grammar (CFG) restricts productions to the form $A \to \alpha$ where $A \in V$ and $\alpha \in (V \cup \Sigma)^*$ [2]. Chomsky's seminal analysis demonstrated that finite-state and simple phrase-structure models are insufficient for natural languages and motivated hierarchical grammatical descriptions that foreshadow modern CFG-based parsing [1].

The Chomsky hierarchy classifies grammars by generative power; programming languages are typically specified by CFGs plus lexical rules [2, 3]. Hopcroft, Motwani, and Ullman provide the standard automata-theoretic treatment of recognition, while Aho, Lam, Sethi, and Ullman (**Compilers: Principles, Techniques, and Tools**) connect grammars to lexical analysis, parsing, and the construction of syntax trees in production compilers [3].

The C0 grammar used in this thesis is a CFG extended with lexical productions for identifiers, digits, and character constants. Unlike English or other natural languages, programming languages are deliberately designed to be unambiguous at the syntactic level; a parser implementation therefore expects a unique derivation structure for each valid program, modulo documented implementation choices.

Derivation Trees, Parse Trees, and Abstract Syntax Trees

A **derivation** is a sequence of production applications transforming the start symbol into a terminal string. A **derivation tree** (parse tree) records this process hierarchically: internal nodes are non-terminals, leaves are terminals, and each parent–child group corresponds to one production's right-hand side [3]. For unambiguous grammars, leftmost and rightmost derivations induce the same tree shape.

Compiler textbooks distinguish parse trees from **abstract syntax trees** (ASTs), which collapse syntactically redundant nodes (e.g., grouping parentheses) to expose program meaning [3]. Benchmarks such as CodeSyntax [20] and CodeBLEU [30] often evaluate structural similarity at the AST level. This thesis deliberately requires **fully expanded derivation trees**: every non-terminal must be resolved to terminal leaves, including letter-by-letter name expansion and digit-by-digit numeric constants. That stricter criterion aligns with how grammar rules are taught in compiler courses and exposes errors that AST-level or token-level metrics would conceal.

Compiler Construction and Pedagogical C0 Dialects

Compiler front ends implement lexical analysis (tokenization) followed by syntactic analysis (parsing) [3]. Pedagogical language subsets reduce semantic complexity so students can focus on parsing and code generation. Carnegie Mellon's C0 was introduced as a safe, contract-oriented imperative language for introductory computation courses [4, 5]. Arnold's technical report documents design goals: eliminate undefined behavior, support formal reasoning, and keep syntax close to C while remaining amenable to automated checking [4].

C0 dialects also appear extensively in Chinese university compiler curricula, where students implement recursive-descent parsers and code generators for course projects [34]. The KIU variant evaluated here—grounded in the reference C0 compiler from a prior KIU bachelor's thesis [33]—extends pedagogical C0 with typedefs, structs, and pointer notation using `'` (dereference) and `&` (address-of). This notation creates a deliberate ambiguity with multiplication (`*`) that tests whether a system applies disambiguation rules rather than surface token co-occurrence statistics.

Probabilistic Models of Code and the Limits of Token-Level Learning

Allamanis, Barr, Devanbu, and Sutton survey the **naturalness** hypothesis: source code, like natural language, exhibits strong statistical regularities that machine learning models can exploit [12]. The survey taxonomy spans n-gram models, tree-based models, and neural architectures, and documents applications from completion to bug detection. However, the authors also emphasize that programming languages possess formal semantics and rigid syntactic constraints that distinguish them from free-form text.

Subsequent benchmarks probe whether pre-trained models internalize these constraints. Feng et al.'s CodeBERT [17] and Kanade et al.'s CuBERT [18], trained on large corpora such as CodeSearchNet [19], achieve strong results on retrieval and documentation tasks. Wang et al.'s CodeSyntax benchmark tells a different story for syntax understanding: when asked to predict syntactic rela-

tionships extracted from AST edges, CodeBERT and CuBERT **underperform** naive keyword-and-position baselines [20]. The authors conclude that massive code pre-training does not reliably encode explicit syntactic structure.

Velasco et al. apply causal analysis (SyntaxEval) to masked language models and report negative causal effects between AST node types and completion accuracy, suggesting that high token prediction scores can overestimate grammatical competence [23]. Ma et al. evaluate 21 LLMs on zero-shot syntax parsing, static analysis, and dynamic reasoning, finding a consistent hierarchy: strong AST-generation performance on simple tasks, weaker static analysis, and limited dynamic reasoning [22]. Together, these studies agree that **functional or token-level success does not imply faithful CFG reasoning**—a premise this thesis tests directly by requiring explicit derivation trees.

Code Generation Benchmarks and What They Measure

The dominant evaluation paradigm for code LLMs measures whether generated programs pass unit tests. Chen et al. introduce Codex and HumanEval (164 Python problems) with the pass@k metric [15]. Austin et al. contribute MBPP (974 crowd-sourced problems), showing that synthesis quality scales with model size but also depends heavily on pre-training data and method [16]. Li et al.'s APPS benchmark extends difficulty to competition-level problems [35]. Liu et al. demonstrate that augmenting HumanEval with stricter testing changes model rankings, underscoring benchmark sensitivity [31].

These benchmarks revolutionized code LLM research but evaluate **outcomes**, not **derivations**. A program may pass tests while being syntactically ill-formed relative to a course grammar, or vice versa. Gong et al.'s SAFIM benchmark moves closer to structure by evaluating syntax-aware fill-in-the-middle completions across control-flow and API-call boundaries [21], yet still measures generated code fragments rather than complete parse trees tied to a fixed production system.

Hou et al.'s systematic literature review of LLMs for software engineering catalogues hundreds of studies but notes that formal grammar compliance and parser-generated artifacts remain under-represented relative to generation and repair tasks [32]. The present thesis therefore complements execution-based benchmarks with a grammar-faithful structural evaluation.

Grammar-Constrained Decoding and Reliable Structured Generation

A parallel research line asks how to **force** LLMs to respect grammars during decoding rather than evaluating free-form outputs post hoc. Poesia et al.'s Synchromesh framework applies Constrained

Semantic Decoding so that partial outputs remain consistent with language-specific syntax and typing rules, demonstrating large reductions in invalid SQL and domain-specific programs without fine-tuning the base model [24]. Willard and Louf reformulate generation as finite-state and push-down-automaton transitions, enabling efficient CFG-guided masking implemented in the Outlines library [25]. Dong et al.'s XGrammar engine optimizes token masking for structured agent outputs at LLM serving time [26]. Ugare et al. and related systems similarly compile grammars into decoders that guarantee syntactically valid JSON or code skeletons [37, 39].

These approaches demonstrate that **external constraints** can enforce grammar compliance when integrated into the inference loop. They do not, however, measure whether an unconstrained model **understands** a grammar when asked to derive trees from rules alone—the setting of this thesis. The unconstrained protocol is deliberately harder and reflects how students or instructors might query an AI assistant without attaching a constrained decoder.

AI Reasoning versus Pattern Matching

Whether LLM behavior constitutes reasoning or sophisticated pattern matching is actively debated. Wei et al. show that chain-of-thought prompting elicits multi-step solutions on arithmetic and commonsense tasks [28], but such prompting was intentionally excluded here to measure baseline competence. Mirzadeh et al.'s GSM-Symbolic study finds high variance when problem templates are perturbed and argues that apparent mathematical reasoning may reflect memorized solution patterns rather than formal inference [27]. Jia and Liang previously showed that adversarial distractor sentences sharply reduce reading comprehension accuracy in neural models, illustrating fragility of pattern-based strategies [29].

Applied to grammars, the concern is analogous: a model may reproduce derivation shapes common in training data without applying the supplied production rules to novel inputs—especially pointer syntax or out-of-grammar constructs. This thesis treats **fully legal derivation** and **stable performance across syntactic categories** as stronger evidence of understanding than high structural similarity on simple programs alone.

Tree Edit Distance and Structural Comparison

Comparing labeled trees requires metrics beyond string edit distance. Tai (1979) introduced the tree-to-tree correction problem with insert, delete, and relabel operations on ordered trees [7]. Zhang and Shasha (1989) gave a faster $O(n^4)$ dynamic programming algorithm that became the standard reference implementation for ordered tree edit distance [8]. Bille's survey classifies subsequent al-

gorithms—including bounded-distance methods of Touzet [40] and worst-case optimal algorithms of Demaine et al. [10]—and documents applications from XML matching to program differencing [9]. Pawlik and Augsten's RTED unifies several strategies and remains among the fastest exact implementations [11].

Selkow (1977) proposed a related ordered editing model with a simpler recursive structure that is sufficient for many practical tree differencing tasks and is implemented in this thesis's comparison tool [6]. The normalized similarity used here,

$$\text{similarity} = \max\left(0, 1 - \frac{d(T_A, T_B)}{|T_A| + |T_B|}\right) \tag{1}$$

where d is Selkow distance and $|T|$ counts nodes, follows common practice in information retrieval and program differencing [6, 9].

Advantages: global structural alignment; tolerance for localized insertions and deletions; continuous scores suitable for aggregation.

Limitations: sensitivity to tree size (spurious large subtrees dominate distance); equal relabeling cost for all symbols; inability to credit alternative but equally valid derivations; dependence on normalization choices [9].

Structural similarity must therefore be paired with a grammar-validity oracle, as in this thesis's `tree_diff_v2.html` implementation.

Related Work on LLM Grammar and Parse-Structure Evaluation

Table Table 1 compares representative prior studies to the present thesis. No prior benchmark identified in this review requires models to emit **complete derivation trees** for a fixed CFG under a standardized graph encoding with compiler-generated ground truth.

Study	Primary artifact	Grammar explicit?	Structural metric
HumanEval [15]	Executable Python	No	pass@k
MBPP [16]	Executable Python	No	pass@k
CodeSyntax [20]	AST edge prediction	Implicit	Classification acc.
SAFIM [21]	Code FIM spans	Implicit	Exact match
Synchromesh [24]	Constrained programs	Yes (decoder)	Task accuracy
Ma et al. [22]	AST/CFG tasks	Partial	Task-specific
This thesis	**Mermaid derivation trees**	**Yes (prompted)**	**Selkow + legality**

Table 1: Comparison of related evaluation paradigms.

The closest methodological neighbors are syntax probes (CodeSyntax, SyntaxEval) and zero-shot parsing evaluations (Ma et al.) [20, 22, 23]. Those works test whether models **encode** or **recover** syntactic relations; they do not require step-by-step exposition of every production in a course grammar. Constrained decoding systems [24, 25, 26] guarantee validity by construction but evaluate engineering mechanisms rather than unconstrained grammatical knowledge.

Gap in the Literature and Contribution of This Thesis

Three gaps motivate this work:

1. **Evaluation target:** Existing benchmarks emphasize executable code or AST relations, not fully expanded derivation trees tied to an explicit CFG.
2. **Ground truth:** Few studies compare LLM outputs to parser-generated reference trees from a real compiler pipeline [33].
3. **Failure analysis:** Pointer disambiguation and out-of-grammar inputs are rarely tested systematically in grammar-centric evaluations.

This thesis contributes:

1. A reproducible protocol (grammar document, Mermaid format standard, 51-program corpus, automated comparison pipeline).
2. Empirical comparison of five contemporary LLMs under identical prompting.
3. Dual metrics combining Selkow structural similarity with a production-rule validity oracle.
4. Category-stratified analysis (standard, pointer, out-of-grammar) linking results to formal understanding versus pattern matching [27, 20].

By positioning derivation-tree generation between compiler theory [3] and LLM code evaluation [12, 15], the study offers a falsifiable test of whether models can act as informal parsers when given explicit rules—a capability increasingly assumed but rarely measured in educational deployments.

3. Methodology

Research Design

The independent variable is the LLM; all other conditions are held constant. The evaluated systems are general-purpose transformer-based models [13, 14] accessed via their public APIs in June 2026.

Experimental Protocol

For each of 51 tasks, every model received (`instruction.txt`):

1. The complete C0 grammar (lexical, expression, statement, and program rules) [3].
2. The C0 Derivation Tree Output Format (v2) specifying Mermaid conventions.
3. One C0 source program.
4. The instruction: output only the Mermaid diagram; do not ask questions; draw the tree as you think it should be.

No model-specific tuning, few-shot examples, temperature adjustment, or follow-up corrections were applied. Each model was queried once per task.

Models Evaluated

Label	Product	Output folder
ChatGPT	GPT-5.5	`GPT_GPT-5.5_answers/`
Gemini	Gemini 3.5 Flash	`Gemini_3.5flash_answers/`
Claude	Claude Sonnet 4.6	`Claude_Sonnet_4.6/`
Grok	Grok 4.6 Fast	`Grok_4_6fast_answers/`
DeepSeek	DeepSeek-V3	`DeepSeek-V3_answers/`

Table 2: AI systems evaluated and output locations. Model versions: OpenAI GPT-5.5 [41], Google Gemini 3.5 Flash [43], Anthropic Claude Sonnet 4.6 [42], xAI Grok 4.6 Fast [44], DeepSeek-V3 [45].

Ground-Truth Generation

Reference derivation trees were produced by the reference C0 compiler from a prior KIU bachelor's thesis [33]. Files in `generated_trees/` pair each test program (`.c0`) with a Mermaid tree (`.mmd`), named with zero-padded task numbers (e.g., `01_single_int_max_constant.mmd`). These reference trees may include ` ` spacing nodes and end-of-input markers absent from the AI format spec; the comparison tool removes these during normalization [6].

Dataset Preparation and Test Design

The test corpus (`accepted C0 lines.txt`) contains 51 programs in three categories:

Category A: Standard C0 Syntax (Tasks 1–44)

Forty-four programs exercising constants, arithmetic, comparisons, arrays, loops, conditionals, recursion, and typedefs—from simple assignments to GCD and power functions.

Category B: Pointer Programs (Tasks 45–48)

Four programs testing pointer declarations (`typedef signed' ptr`), address-of (`b&`), dereference (`ptr'`, `a'.val`), struct pointers, and linked-list construction. These probe whether models distinguish `'` (dereference) from `*` (multiplication or pointer type).

Category C: Out-of-Grammar Programs (Tasks 49–51)

Three programs with constructs absent from the C0 grammar: inline assembly `asm(...)`, general-purpose register access `gpr(1)`, and combined usage. These test whether models refuse invalid input or invent productions.

Category	Count
Standard in-grammar (Tasks 1–44)	44
Pointer-focused (Tasks 45–48)	4
Out-of-grammar (Tasks 49–51)	3
Total	**51**

Table 3: Test corpus distribution.

Prompt Engineering

The prompt bundle deliberately minimizes ambiguity while avoiding examples that could leak reference structures:

- Grammar and format rules are provided verbatim.
- Models are told structure comes entirely from the grammar; the format document only fixes encoding.
- Terminal values must be separate leaf nodes; names and numbers fully expanded.
- The closing line discourages refusal: “don’t ask questions just draw the tree.”

This design tests whether models comply with formal constraints under pressure to produce output for every input, including invalid programs.

Evaluation Metrics

Two metric families are computed by `tree_diff_v2.html`:

Grammar Validity (Reference-Independent)

- **legality:** Fraction of internal nodes whose children match a valid C0 production.
- **fullyLegal:** Boolean; all internal nodes legal.
- **invented, noProduction, unexpanded, collapsedLeaves:** Error counts by type.

Structural Similarity (Reference-Dependent)

- **structural:** Normalized Selkow similarity per Equation 1 [6, 9].
- **editDistance:** Raw Selkow distance.
- **border:** Similarity of leaf sequences (derived program string).
- **edge, node:** Dice coefficients over production pairs and node label bags.
- **derivesProgram:** Whether border similarity ≥ 0.999.

Normalization options (all enabled): flatten sequence non-terminals, merge collapsed leaves, remove noise nodes, match terminal values in signatures.

Tree Comparison Process

A Python script (`run_tree_diff_v2.py`) automates batch comparison via Playwright: for each task, it loads reference and candidate `.mmd` files into `tree_diff_v2.html`, triggers comparison, and appends one CSV row per task to `results/`. The dashboard (`c0_derivation_tree_thesis_dashboard.html`) visualizes the same CSV data. Figures in Chapter 5 are exported from the dashboard to `figures/` (adapted for print layout).

4. Implementation

Mermaid Derivation Tree Structure

Each tree is a directed graph in Mermaid `graph TD` syntax. Nodes use the form `id["label"]` with unique identifiers (ignored during comparison). Non-terminal labels are HTML-escaped angle-bracket symbols (`<E>`). Terminal labels are literal tokens. Edges `parent --> child` encode derivation; child order follows production right-hand side order. The root is always `<prog>`.

Example fragment for deriving the letter `a` through `<Le>`:

```
n7["&lt;Le&gt;"]
n8["a"]
n7 --> n8
```

Incorrect encodings embed values in non-terminal labels (`<Le> = a`) or skip lexical expansion steps; the grammar oracle flags these as `collapsedLeaves` or `unexpanded` violations.

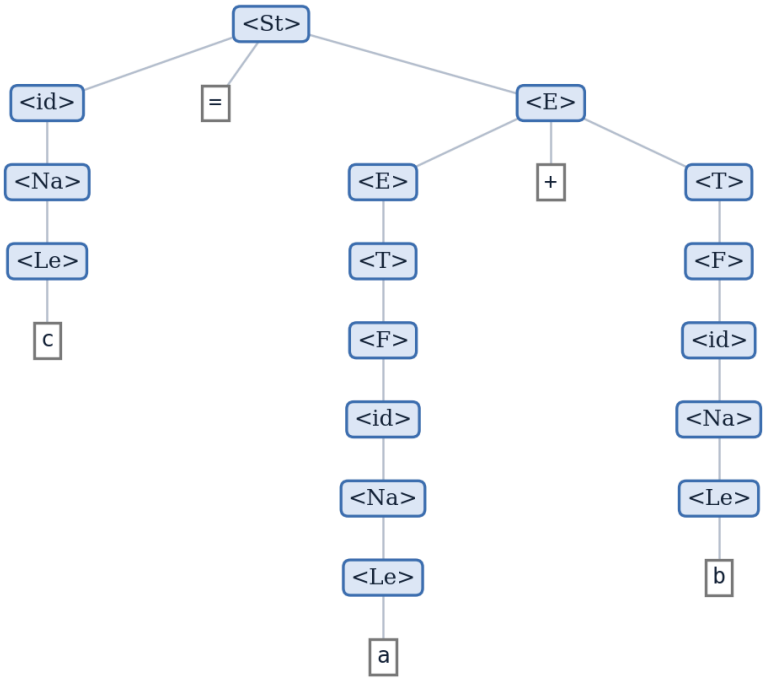

Figure 1: Derivation tree for the statement `c = a + b` *under the C0 grammar. Non-terminals (blue) expand down to terminal leaves (grey). Note the left-recursive* `<E> -> <E> + <T>` *production and the full lexical expansion of each identifier through* `<id> -> <Na> -> <Le>`*—the level of detail every model is required to reproduce.*

Reference Compiler Outputs

The reference compiler emits one Mermaid file per parsed program. Reference trees in `generated_trees/` contain the authoritative derivation paths chosen by the compiler's parser. File naming maps task numbers to descriptive slugs (e.g., Task 9 → `09_single_int_arithmetic_add.mmd`). Plain-text tree dumps (`.tree.txt`) are also available for manual inspection.

Tree Comparison Tool (`tree_diff_v2.html`)

The comparison pipeline proceeds in five stages:

1. **Parse:** Extract nodes and edges from Mermaid text; classify each node as non-terminal (`<Sym>`), terminal literal, collapsed non-terminal leaf, or noise.
2. **Normalize:** Optionally remove noise (` `, ~), merge collapsed leaves, and flatten left-recursive sequence non-terminals (`StS`, `VaDS`, `DiS`, etc.).
3. **Validate:** Walk the candidate tree; check each internal node against the C0 production table encoded in JavaScript (`PRODUCTIONS`, `SEQ_DEF`).
4. **Compare:** Compute Selkow distance and auxiliary metrics (border, edge, node Dice coefficients).
5. **Report:** Display gauges, issue lists, side-by-side rendered trees, and CSV export.

Selkow Algorithm Implementation

The `ted(A, B, vals)` function implements ordered tree edit distance with memoization, following Selkow's editing model [6]. Zhang and Shasha's algorithm [8] offers superior asymptotic bounds for large trees but was not required given derivation-tree sizes in this corpus (typically fewer than 500 nodes per reference tree). For trees rooted at nodes a and b:

1. **Relabeling cost:** 0 if node signatures match (non-terminal symbol or terminal value), else 1.
2. **Forest alignment:** Let m and n be child counts. Dynamic programming table $D[i][j]$ stores minimum cost to align the first i children of a with the first j children of b:
   - Deletion: $D[i-1][j] + |T_{\{a,i\}}|$ (subtree size of i-th child of a)
   - Insertion: $D[i][j-1] + |T_{\{b,j\}}|$
   - Match/substitute: $D[i-1][j-1] + d(a_i, b_j)$
3. **Total distance:** $d(a, b) = \text{relabel}(a, b) + D[m][n]$.

The structural similarity reported in results is:

$$\text{structural} = \max\left(0, 1 - \frac{\text{ted}(T_{\text{ref}}, T_{\text{cand}})}{|\text{ref}| + |\text{cand}|}\right) \tag{2}$$

where $|\text{ref}|$ and $|\text{cand}|$ are total node counts after normalization.

The grammar oracle operates independently on the candidate tree, enabling analysis of cases where legality is high but structural similarity is low (alternative valid derivations or invented paths for invalid inputs).

Result Processing Pipeline

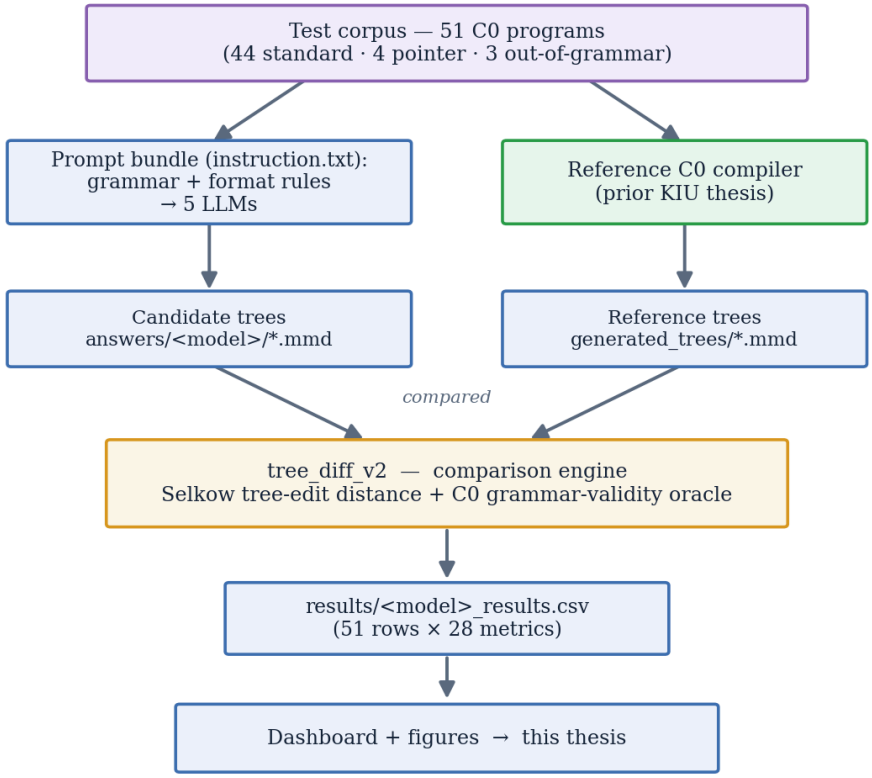

Figure 2: End-to-end experimental pipeline. Each C0 program is sent to the five models together with the grammar and format rules (`instruction.txt`) and, in parallel, parsed by the reference compiler. The candidate and reference trees are then compared by `tree_diff_v2`—which combines Selkow tree-edit distance with a C0 grammar-validity oracle—to produce the per-model result tables and figures.

Each CSV row records task id, file names, timestamp, all validity counts, structural metrics, node counts, edit distance, and normalization flags.

5. Results

Quantitative results were extracted from per-model CSV files in `results/` (June 17, 2026). Structural similarity is the primary metric for RQ1 and RQ2; legality and error counts support interpretive analysis.

Overall Structural Similarity

Model	Mean	Median	Min	Max	Stdev
Claude Sonnet 4.6	0.864	0.890	0.545	1.000	0.112
Gemini 3.5 Flash	0.831	0.970	0.176	1.000	0.262
ChatGPT (GPT-5.5)	0.830	0.923	0.306	1.000	0.195
Grok 4.6 Fast	0.730	0.722	0.359	1.000	0.177
DeepSeek-V3	0.313	0.314	0.006	0.655	0.190

Table 4: Structural similarity statistics across all 51 tasks ($n = 51$ per model).

Claude Sonnet 4.6 ranks first on mean (0.864), median (0.890), minimum (0.545), and standard deviation (0.112, lowest variance among top performers). Gemini produced the most fully legal trees (10 of 51, all on simple constant and arithmetic programs) and frequently reached perfect structural similarity on them, but also recorded the lowest single-task minimum (0.176, on the out-of-grammar Task 50). DeepSeek-V3 clusters near 0.31 with catastrophic failures on complex tasks (e.g., Task 35: structural 0.006, 70,070 candidate nodes vs. 241 reference nodes).

Overall Model Performance Summary

Model	Mean struct.	Mean legality	Mean edit dist.	Fully legal	Border-correct
Claude Sonnet 4.6	0.864	0.947	42.4	1	6
Gemini 3.5 Flash	0.831	0.946	69.5	10	13
ChatGPT (GPT-5.5)	0.830	0.922	55.3	4	23
Grok 4.6 Fast	0.730	0.838	85.1	2	20
DeepSeek-V3	0.313	0.776	2715.5	0	0

Table 5: Overall model performance across all 51 tasks (computed from `results/`). Border-correct counts tasks whose derived program string matches the reference, i.e. border similarity ≥ 0.999.

Key aggregate findings (all 51 tasks):

- **Highest mean structural similarity:** Claude (0.864).
- **Highest mean legality:** Claude (0.947) and Gemini (0.946).

- **Lowest mean edit distance:** Claude (42.4).
- **Most fully legal trees:** Gemini—10 of 51 (all on simple constant and arithmetic tasks)—followed by ChatGPT (4: Tasks 22, 26, 27, 49), Grok (2), and Claude (1); DeepSeek-V3 produced none.
- **Most border-correct derivations (**border ≥ 0.999**):** ChatGPT (23), Grok (20), Gemini (13), Claude (6), DeepSeek (0).
- **Worst performer:** DeepSeek-V3 (mean structural 0.313, mean edit distance 2715.5, 16,167 illegal nodes total).

Performance by Test Category

Model / Category	Avg structural	Avg legality	Avg edit dist.	Fully legal
Claude / Normal	0.888	0.950	27.5	1 / 44
Claude / Pointer	0.772	0.941	157.2	0 / 4
Claude / OOG	0.640	0.918	107.0	0 / 3
Gemini / Normal	0.908	0.964	30.0	10 / 44
Gemini / Pointer	0.284	0.767	427.2	0 / 4
Gemini / OOG	0.428	0.925	172.0	0 / 3
ChatGPT / Normal	0.883	0.946	35.1	3 / 44
ChatGPT / Pointer	0.502	0.631	217.8	0 / 4
ChatGPT / OOG	0.486	0.950	135.0	1 / 3
Grok / Normal	0.766	0.865	63.4	2 / 44
Grok / Pointer	0.421	0.540	309.2	0 / 4
Grok / OOG	0.599	0.854	104.7	0 / 3
DeepSeek / Normal	0.312	0.780	3037.6	0 / 44
DeepSeek / Pointer	0.411	0.693	873.8	0 / 4
DeepSeek / OOG	0.201	0.830	446.0	0 / 3

Table 6: Category breakdown computed from CSV results files.

On standard tasks, Gemini leads structural similarity (0.908); Claude is close (0.888) with lower edit distance. Pointer tasks reduce all models' scores; no model produced a fully legal pointer tree. Out-of-grammar tasks yield moderate legality (models invent plausible rules) but low structural alignment with reference trees (which also cannot parse these inputs faithfully).

Error Type Totals

Model	Invented	No production	Unex-panded	Total illegal
Claude Sonnet 4.6	0	323	12	335
Gemini 3.5 Flash	7	245	208	460
ChatGPT (GPT-5.5)	0	337	8	345
Grok 4.6 Fast	0	742	247	989
DeepSeek-V3	0	14,006	2,161	16,167

Table 7: Aggregate grammar-violation counts by type across all 51 tasks per model.

Gemini is the only model that invented non-C0 non-terminal symbols (7 instances). DeepSeek's error totals (16,167 illegal nodes) exceed other models by two orders of magnitude, dominated by `noProduction` (14,006) and `unexpanded` (2,161) violations—systematic failure to follow the expansion rules.

Structural Similarity by Category

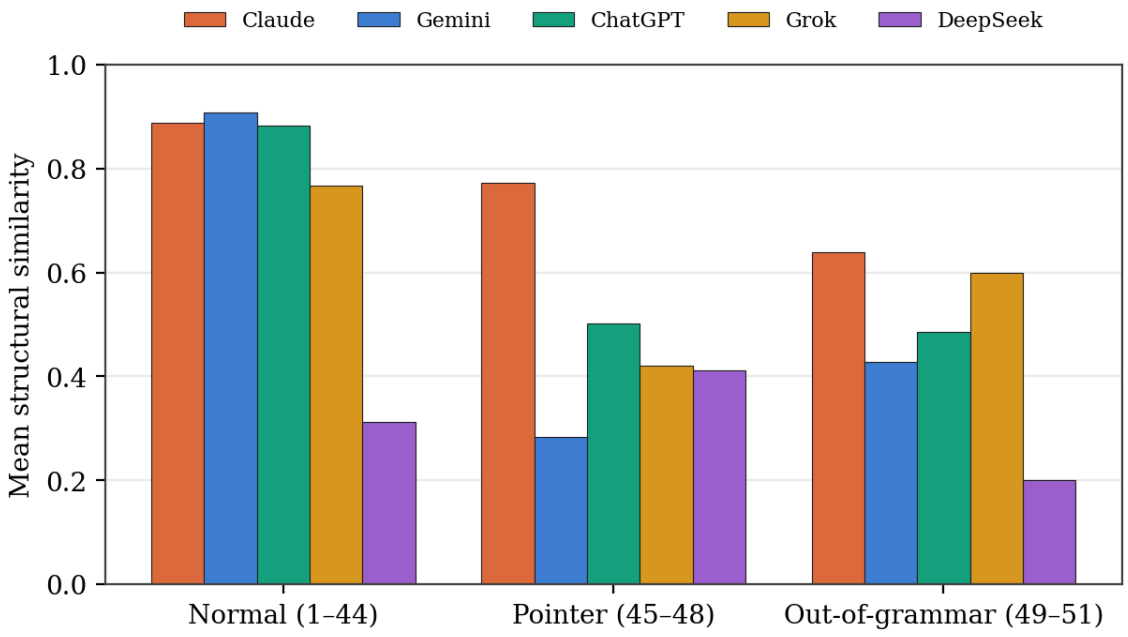

Figure 3: Average structural similarity by test category for all five models (adapted from the results dashboard).

Normal tasks cluster above 0.76 for four models (excluding DeepSeek). Pointer tasks show the steepest drop for Gemini (0.908 → 0.284). Claude remains most stable on pointers (0.772).

Edit Distance by Category

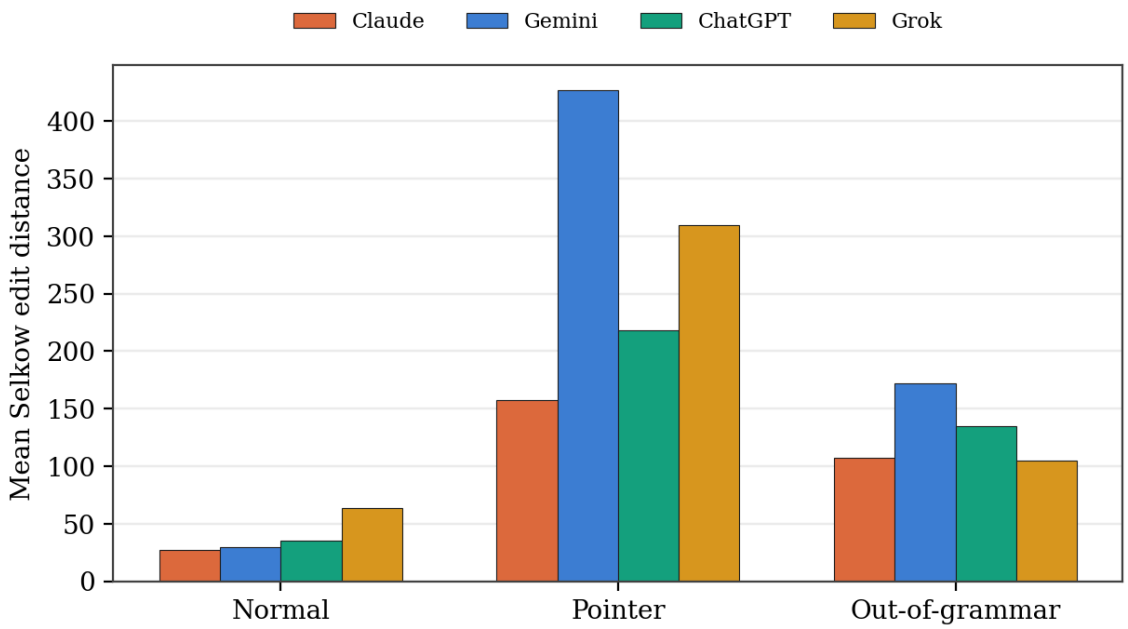

Figure 4: Average Selkow edit distance by category; DeepSeek-V3 omitted for scale (adapted from the results dashboard).

Pointer tasks inflate edit distance for all models. Gemini's pointer average (427.2) exceeds Claude's (157.2) by nearly 3× despite higher normal-task performance.

Legality versus Structural Similarity

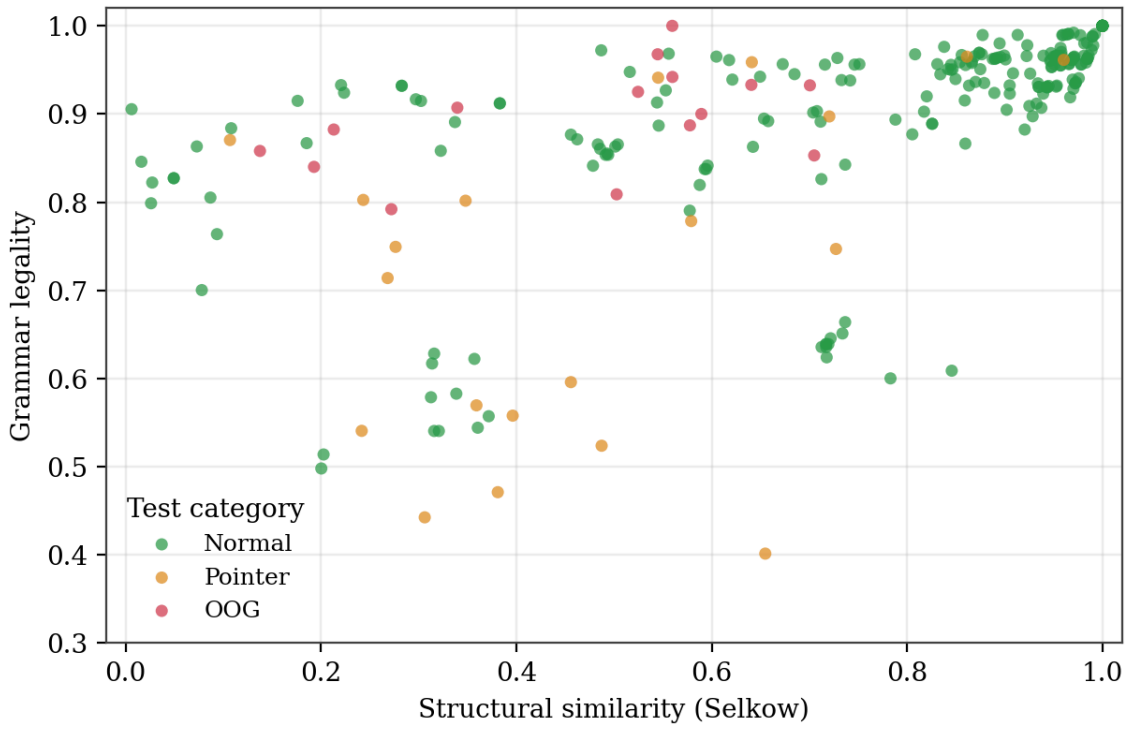

Figure 5: Grammar legality versus structural similarity for all 255 task–model pairs, coloured by test category (adapted from the results dashboard).

Normal tasks (green) cluster upper-right. Pointer and out-of-grammar tasks often show high legality with low structural similarity—the **OOG paradox**: invented or alternate derivations can be locally grammatical yet structurally divergent from the reference.

Per-Task Structural Heatmap

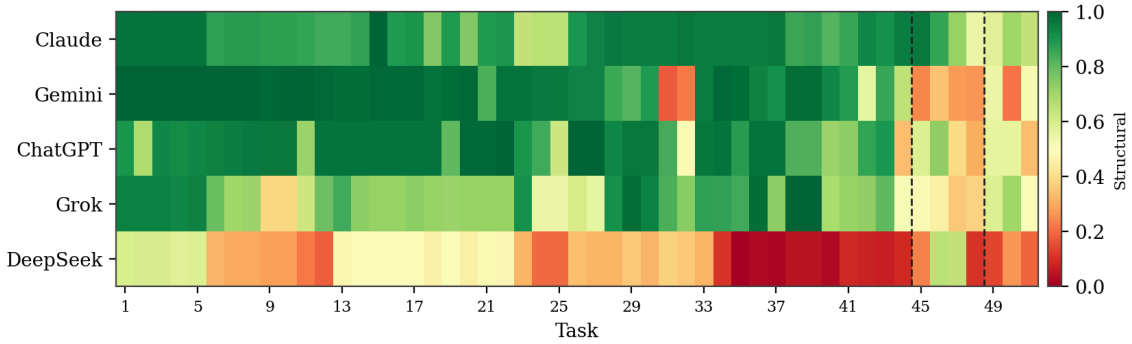

Figure 6: Per-task structural similarity heatmap (5 models × 51 tasks); dashed lines mark the pointer and out-of-grammar blocks (adapted from the results dashboard).

Tasks 45–48 (pointer block) and 49–51 (OOG) form consistent cold zones. DeepSeek's row is predominantly low across all tasks. Task 35 (GCD while-loop) shows near-zero scores for DeepSeek due to runaway expansion.

Model Profiles on Normal Tasks

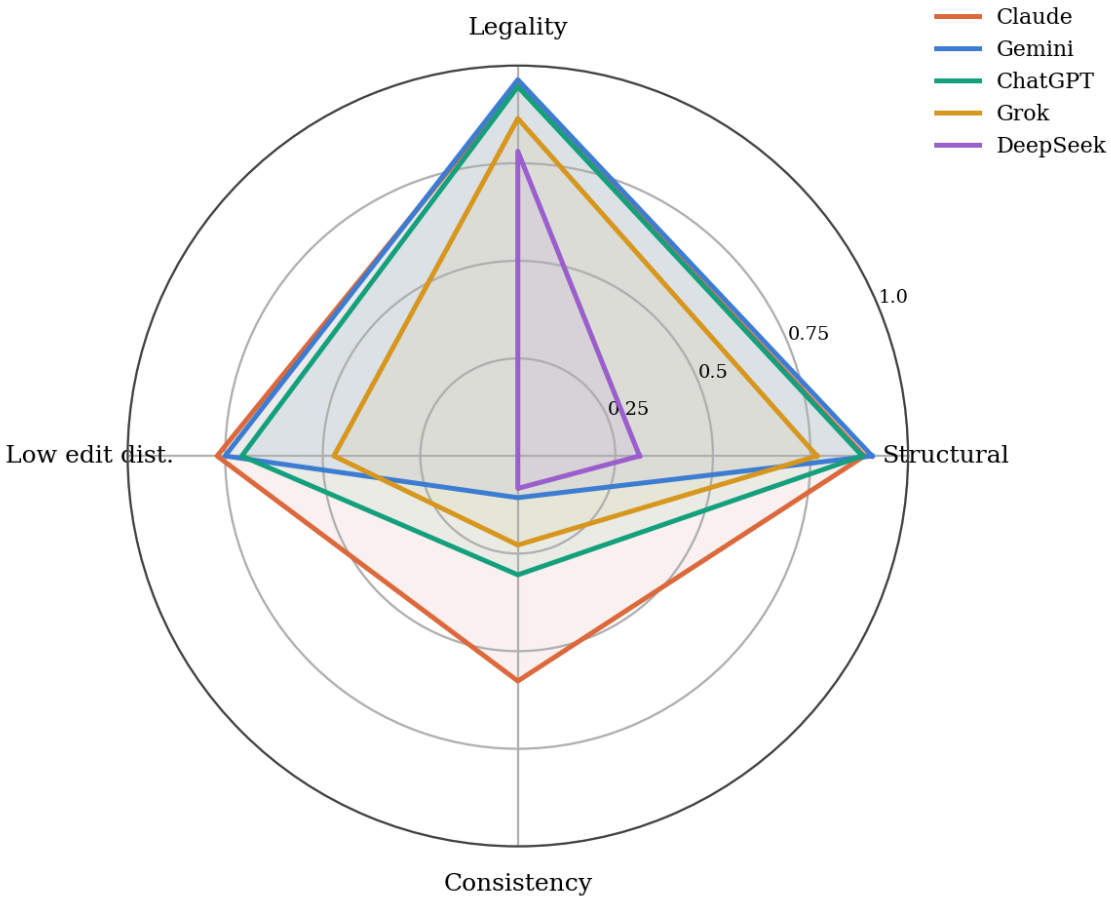

Figure 7: Normalized model profiles on standard (in-grammar) tasks across structural similarity, legality, edit-distance economy, and consistency (adapted from the results dashboard).

Most and Least Challenging Tasks

Task	Program type	Avg legality (5 models)
48	Linked-list struct pointers	0.688
46	Simple pointer dereference	0.702
45	Struct pointer equality	0.721
47	Struct pointer with pow()	0.746
25	Complex uint arithmetic	0.841

Table 8: Five most challenging tasks (lowest average legality).

Task	Program type	Avg legality (5 models)
35	While-loop GCD	0.958
10	Signed subtraction	0.954
34	While-loop division	0.951
9	Signed addition	0.949
8	Boolean false constant	0.944

Table 9: Five easiest tasks (highest average legality).

Notable Individual Results

- **Task 22 (ChatGPT):** Perfect metrics—legality 1.0, structural 1.0, edit distance 0, fully legal.
- **Tasks 26–27 (ChatGPT):** Perfect structural match on boolean comparisons.
- **Tasks 38–39 (Grok):** Structural 1.0 on if-else min/max programs.
- **Task 44 (GCD):** High edit distances—ChatGPT 448, Grok 328, DeepSeek 3058.
- **Task 49 (ChatGPT):** Fully legal tree for `asm()` input despite construct being out-of-grammar (legality 1.0 via invented valid-looking structure).

6. Discussion

Answers to Research Questions

RQ1 (Can LLMs correctly generate derivation trees for valid C0 programs?): Partially. On standard tasks (1–44), mean structural similarity ranges from 0.312 (DeepSeek) to 0.908 (Gemini). No model achieved fully legal trees on a majority of tasks. Gemini produced the most fully legal trees (10, all on simple constant and arithmetic tasks), followed by ChatGPT (4, one of them an out-of-grammar case); no model produced a fully legal tree on any pointer task. Models frequently generate trees with correct leaf sequences but invalid internal nodes, indicating surface-level rather than complete grammatical derivation.

RQ2 (Which model is most similar to the reference compiler?): Claude Sonnet 4.6 ranks first on mean structural similarity (0.864) and lowest edit distance (42.4). Gemini leads on normal-task structural similarity (0.908) but with higher variance. Claude provides the best overall balance of structural alignment and stability across categories.

RQ3 (Pointer vs. multiplication disambiguation?): Models struggle substantially. Pointer task structural similarity ranges from 0.284 (Gemini) to 0.772 (Claude). Grok shows the weakest pointer legality (0.540). Common errors include applying `<T> -> <T> * <F>` to pointer dereference contexts or mishandling `'` and `&` in `<id>` productions. Claude demonstrates the best pointer handling; Gemini's large normal/pointer gap (0.908 vs. 0.284) suggests pointer syntax requires qualitatively different reasoning.

RQ4 (Response to invalid inputs?): No model refused to generate a tree. All attempted derivations for Tasks 49–51, inventing structures for `asm()` and `gpr()`. ChatGPT achieved fully legal status on Task 49 despite invalid input. Legality remains deceptively high (0.83–0.95 across models on OOG tasks) because invented nodes can satisfy local production patterns. Structural similarity to reference trees stays low (0.20–0.64), confirming that neither models nor references can faithfully parse these programs under the C0 grammar.

RQ5 (Does high similarity imply deep understanding?): Not necessarily. High structural similarity correlates with adherence to expected derivation paths on training-like constructs, but legality and structural scores diverge: Grok achieves border-correct leaf sequences on 20 normal tasks with zero fully legal trees. ChatGPT derives correct program text in 23 cases while only 4 trees are fully legal. High similarity on simple tasks combined with collapse on pointers and invented rules for OOG inputs indicates pattern recognition over explicit grammar rule application. True

understanding would imply consistent fully legal trees, reliable rejection of ungrammatical input, and stable performance across syntactic categories.

Why Some Models Performed Better

Claude's advantage likely stems from stronger instruction-following for hierarchical structured output and more conservative tree expansion, yielding lower edit distance and smaller illegal-node counts. Gemini excels on familiar in-grammar patterns (high normal-task similarity) but fails to generalize pointer notation. ChatGPT optimizes for deriving the correct terminal string (border metric) even when internal structure is wrong—useful for program text recovery but insufficient for formal grading. DeepSeek's runaway expansion suggests difficulty controlling recursive production application under length pressure.

Common Error Patterns

1. **Incomplete lexical expansion:** `main` not derived letter-by-letter.
2. **Collapsed leaves:** Terminals embedded in non-terminal labels.
3. **Skipped hierarchy levels:** Jumping from `<E>` to terminals without `<T>`, `<F>`.
4. **Pointer confusion:** Misclassifying `'` and `&` tokens.
5. **Runaway recursion:** Massive trees from repeated production application.
6. **Invented symbols:** Non-grammar non-terminals for `asm`, `gpr` (Gemini: 7 cases).

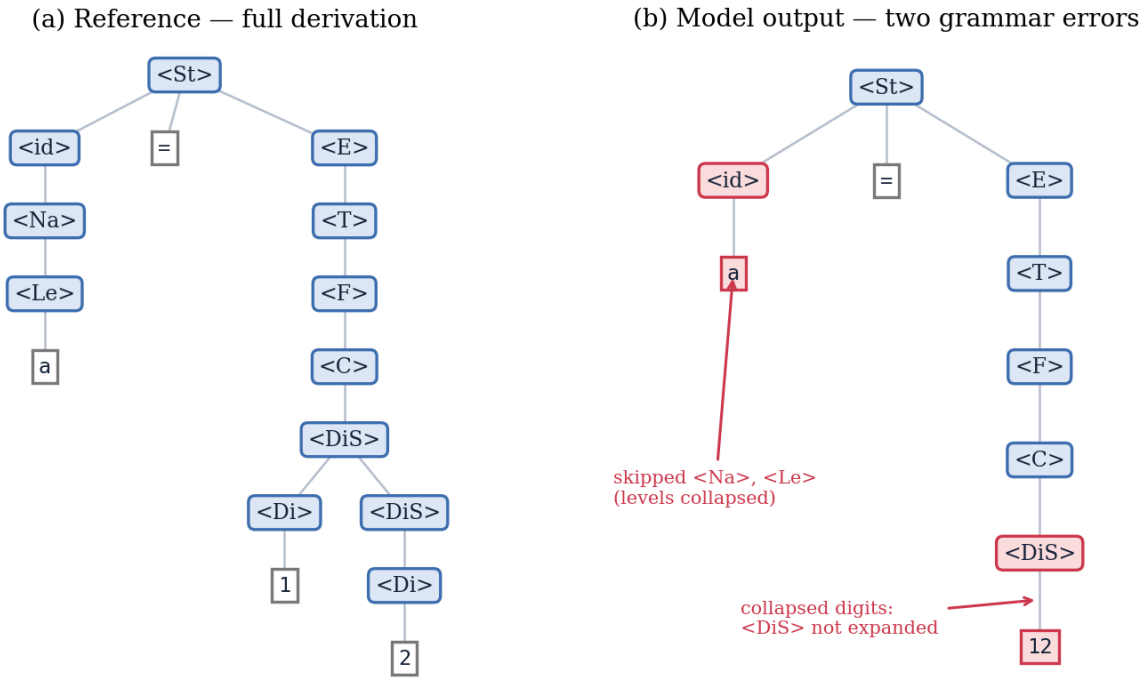

Figure 8: Two recurring failure modes, illustrated on the statement `a = 12`. *(a) The reference fully expands every non-terminal, deriving the name through* `<id> -> <Na> -> <Le>` *and the number digit-by-digit through* `<DiS>`. *(b) A typical model output skips the* `<Na>`/`<Le>` *levels under* `<id>` *and collapses the multi-digit number into a single leaf—both flagged by the grammar oracle as* `unexpanded`/`collapsedLeaves` *violations even though the spelled program is unchanged.*

Comparison with Prior LLM Evaluation Paradigms

Relative to execution-based benchmarks [15, 16, 35], the present study trades functional completeness for grammatical transparency. A model may achieve high pass@k on HumanEval while failing to expand `<DiS>` digit-by-digit, as observed for several models here. Relative to syntax probes [20, 23], this evaluation requires **generating** entire trees rather than classifying AST relations, a strictly harder task. Relative to constrained decoding research [24, 25, 26], models receive no decoder-side grammar enforcement; failures therefore reflect unconstrained competence.

ChatGPT's comparatively high border-correct derivation count (23 programs) echoes findings that LLMs often preserve surface token sequences even when internal structure is wrong [22, 31]—analogous to CodeSyntax models matching keywords without faithful syntax graphs [20].

Genuine Understanding versus Pattern Matching

The evidence favors statistical pattern matching supplemented by format heuristics over explicit grammar manipulation [27, 29]. Models produce Mermaid graphs resembling parse trees seen in train-

ing data but do not consistently apply the provided production set. The disconnect between border correctness and legality, the OOG paradox, and pointer-specific collapse mirror Mirzadeh et al.'s observation that small input perturbations destabilize apparently competent reasoning [27], and Velasco et al.'s finding that syntax-aware causal effects can be weak or negative for code LMs [23]. These patterns are hallmarks of approximate syntactic modeling rather than verified CFG parsing [3].

7. Threats to Validity

Internal Validity

Limited test cases: 51 programs, with only 4 pointer and 3 OOG cases, cannot exhaust the C0 grammar. Results may not generalize to unseen constructs.

Single run per task: No measurement of stochastic variance; a model might succeed on retry.

Prompt sensitivity: The instruction to never refuse input biases models toward forced derivation, potentially understating refusal capability. Grammar-constrained decoders [25, 37] and explicit error-reporting prompts could yield different refusal rates on Tasks 49–51.

Model versioning: Results reflect June 2026 model snapshots; updates may change performance, as benchmark rankings have shifted with model generations [31].

Construct Validity

Mermaid as proxy: Trees are compared after parsing Mermaid syntax, not direct grammar objects. Malformed Mermaid or idiosyncratic node ordering could affect scores independent of grammatical knowledge.

Similarity metric limitations: Selkow distance [6] penalizes alternative valid derivations equally, as documented in tree-edit surveys [9]. A grammatically correct but non-canonical tree scores lower than warranted. Zhang and Shasha's metric [8] would behave similarly with respect to derivation ambiguity.

Legality oracle coverage: The JavaScript production table may not capture every edge case of the prose grammar; discrepancies between oracle and human judgment are possible, as noted in prior grammar-constrained systems [24].

External Validity

Single language: C0-specific syntax (especially `'` pointers) limits generalization to other languages.

Educational subset: C0 is pedagogical; industrial grammars are larger and more ambiguous.

Conclusion Validity

Ground-truth dependence: Reference trees reflect the reference parser's derivation choices. Different parser implementations might choose alternate valid derivations, affecting structural scores without indicating model error.

Normalization effects: Flattening and noise removal align reference and candidate trees but alter raw structures; sensitivity to these options was not fully ablated.

Despite these threats, the controlled multi-model design, dual metrics (structural + legality), and category-stratified analysis provide a credible snapshot of current LLM grammatical competence on a well-defined formal task.

8. Conclusion

Summary of Findings

This thesis evaluated five large language models on C0 derivation tree generation under identical conditions. Key findings:

1. No model achieves reliable formal correctness; at most 10 of 51 trees were fully legal (Gemini), and only on the simplest programs.
2. Claude Sonnet 4.6 produces the most reference-aligned trees overall (mean structural 0.864).
3. Gemini leads on standard-syntax structural similarity (0.908) but fails on pointers (0.284).
4. Pointer tasks are the primary discriminator; Claude handles them best.
5. Out-of-grammar inputs are not rejected; models invent derivations.
6. DeepSeek-V3 is unsuitable for this task without substantial prompting changes.

Research Contributions

1. A reproducible protocol for evaluating LLM grammar understanding via derivation trees rather than executable code.
2. A Mermaid-based encoding standard enabling automated comparison.
3. An open pipeline (`tree_diff_v2.html`, `run_tree_diff_v2.py`, dashboard) combining Selkow distance with a C0 grammar oracle.
4. Empirical comparison of five major LLMs on 51 stratified C0 programs with published CSV results.

Recommendations for Future Work

1. Expand pointer and struct test coverage; add ambiguous expression cases.
2. Run multiple samples per task to quantify variance.
3. Compare prompting strategies (few-shot, chain-of-thought, explicit refusal instructions).
4. Test whether fine-tuning on grammar specifications improves legality.
5. Extend methodology to additional formal grammars, parser generators, and repository-scale benchmarks [36].
6. Integrate the comparison tool into compiler course auto-grading workflows.

Formal grammars demand precision that current LLMs only approximate. Until fully legal derivation becomes the norm rather than the exception, AI-generated parse trees require automated verification before use in education or tooling.

References

[1] Chomsky, N. (1956). Three models for the description of language. **IRE Transactions on Information Theory**, 2(3), 113–124.

[2] Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2006). **Introduction to Automata Theory, Languages, and Computation** (3rd ed.). Pearson.

[3] Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). **Compilers: Principles, Techniques, and Tools** (2nd ed.). Pearson.

[4] Arnold, R. (2010). **C0, an imperative programming language for novice computer scientists** (Technical Report CMU-CS-10-145). Carnegie Mellon University.

[5] Pfenning, F., & Ahmed, A. (2019). Teaching the principles of programming languages with C0. In **Proceedings of the 4th ACM SIGPLAN International Symposium on Programming Languages and Systems Education (PLE)**.

[6] Selkow, S. M. (1977). The tree-to-tree editing problem. **Information Processing Letters**, 6(6), 184–186. https://doi.org/10.1016/0020-0190(77)90064-3

[7] Tai, K.-C. (1979). The tree-to-tree correction problem. **Journal of the ACM**, 26(3), 422–433. https://doi.org/10.1145/322139.322143

[8] Zhang, K., & Shasha, D. (1989). Simple fast algorithms for the editing distance between trees and related problems. **SIAM Journal on Computing**, 18(6), 1245–1262.

[9] Bille, P. (2005). A survey on tree edit distance and related problems. **Theoretical Computer Science**, 337(1–3), 217–239.

[10] Demaine, E. D., Mozes, S., Rossman, B., & Weimann, O. (2009). An optimal decomposition algorithm for tree edit distance. **ACM Transactions on Algorithms**, 6(1), Article 2.

[11] Pawlik, M., & Augsten, N. (2011). RTED: A robust algorithm for the tree edit distance. **Proceedings of the VLDB Endowment**, 5(4), 334–345.

[12] Allamanis, M., Barr, E. T., Devanbu, P., & Sutton, C. (2018). A survey of machine learning for big code and naturalness. **ACM Computing Surveys**, 51(4), Article 81. https://doi.org/10.1145/3212695

[13] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. In **Advances in Neural Information Processing Systems 30 (NeurIPS)**.

[14] Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., et al. (2020). Language models are few-shot learners. In **Advances in Neural Information Processing Systems 33 (NeurIPS)**.

[15] Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. O., Kaplan, J., et al. (2021). Evaluating large language models trained on code. **arXiv preprint arXiv:2107.03374**. https://doi.org/10.48550/arXiv.2107.03374

[16] Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., et al. (2021). Program synthesis with large language models. **arXiv preprint arXiv:2108.07732**.

[17] Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., et al. (2020). CodeBERT: A pre-trained model for programming and natural languages. In **Findings of the Association for Computational Linguistics: EMNLP 2020**.

[18] Kanade, A., Maniatis, P., Balog, M., & Shi, K. (2020). Learning and evaluating contextual embedding of source code. In **Proceedings of the 37th International Conference on Machine Learning (ICML)**.

[19] Husain, H., Wu, H.-H., Gazit, T., Allamanis, M., & Brockschmidt, M. (2019). CodeSearchNet challenge: Evaluating the state of semantic code search. **arXiv preprint arXiv:1909.09436**.

[20] Wang, Y., Wang, W., Joty, S., & Hoi, S. C. H. (2022). Benchmarking language models for code syntax understanding. In **Findings of the Association for Computational Linguistics: EMNLP 2022**.

[21] Gong, L., Wang, W., Li, Y., Zhang, H., & Liu, T. (2024). Evaluation of LLMs on syntax-aware code fill-in-the-middle tasks. In **Proceedings of the 41st International Conference on Machine Learning (ICML)**, PMLR 235:15907–15928.

[22] Ma, W., Lin, Z., Liu, S., Hu, Q., Liu, Y., Wang, W., et al. (2024). Exploring code analysis: Zero-shot insights on syntax and semantics with LLMs. **ACM Transactions on Software Engineering and Methodology**, 33(8).

[23] Velasco, A., Palacio, D. N., Rodriguez-Cardenas, D., & Poshyvanyk, D. (2024). Which syntactic capabilities are statistically learned by masked language models for code? In **Proceedings of the 46th International Conference on Software Engineering (ICSE-NIER)**. https://doi.org/10.1145/3639476.3639768

[24] Poesia, G., Polozov, O., Le, V., Tiwari, A., Soares, G., Meek, C., & Gulwani, S. (2022). Synchromesh: Reliable code generation from pre-trained language models. In **Proceedings of the 10th International Conference on Learning Representations (ICLR)**.

[25] Willard, B. T., & Louf, R. (2023). Efficient guided generation for large language models. **arXiv preprint arXiv:2307.09702**.

[26] Dong, Y., Ruan, C. F., Cai, Y., Lai, R., Xu, Z., Zhao, Y., & Chen, T. (2025). XGrammar: Flexible and efficient structured generation engine for large language models. **Proceedings of Machine Learning and Systems**, 7.

[27] Mirzadeh, I., Alizadeh, K., Shahrokhi, H., Tuzel, O., Bengio, S., & Farajtabar, M. (2024). GSM-Symbolic: Understanding the limitations of mathematical reasoning in large language models. **arXiv preprint arXiv:2410.05229**.

[28] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., et al. (2022). Chain-of-thought prompting elicits reasoning in large language models. In **Advances in Neural Information Processing Systems 35 (NeurIPS)**.

[29] Jia, R., & Liang, P. (2017). Adversarial examples for evaluating reading comprehension systems. In **Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing (EMNLP)**.

[30] Ren, S., Guo, D., Lu, S., Zhou, L., Liu, S., Tang, D., et al. (2020). CodeBLEU: A method for automatic evaluation of code synthesis. **arXiv preprint arXiv:2009.10297**.

[31] Liu, J., Xia, C. S., Wang, Y., & Zhang, L. (2023). Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation. In **Advances in Neural Information Processing Systems 36 (NeurIPS)**.

[32] Hou, X., Zhao, Y., Liu, Y., Yang, Z., Wang, K., Li, L., et al. (2024). Large language models for software engineering: A systematic literature review. **ACM Transactions on Software Engineering and Methodology**, 33(8).

[33] Shilakadze, M., Machavariani, K., & Kvinikadze, N. (n.d.). A C0 compiler and derivation-tree generator [Computer software]. Bachelor's thesis, Kutaisi International University. https://github.com/System-Architecture-WJP/WARS

[34] Maekawa, R. (2019). Chigusa: A C0 compiler for compiler design coursework. Beihang University. https://github.com/lynzrand/chigusa

[35] Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, E., Song, D., & Steinhardt, J. (2021). Measuring coding challenge competence with APPS. **arXiv preprint arXiv:2105.09938**.

[36] Jimenez, C. E., Yang, J., Wettig, A., Yao, K., Pei, K., Press, O., & Narasimhan, K. (2024). SWE-bench: Can language models resolve real-world GitHub issues? In **Proceedings of the 12th International Conference on Learning Representations (ICLR)**.

[37] Ugare, I., Yun, S., Qi, H., & Chaudhuri, S. (2024). SynCode: LLM generation with grammar masking. **arXiv preprint arXiv:2406.04935**.

[38] Beurer-Kellner, L., Fischer, M. N., & Polozov, O. (2024). Guiding LLMs with guidance programs. **arXiv preprint arXiv:2405.19415**.

[39] Geng, S., Konishi, T., Ye, X., Fu, J., & Bansal, M. (2025). JSONSchemaBench: A rigorous benchmark of structured outputs for language models. **arXiv preprint arXiv:2501.10828**.

[40] Touzet, H. (2005). On some combinatorial problems of edit distance algorithms. In **Proceedings of the 16th Annual Symposium on Combinatorial Pattern Matching (CPM)**.

[41] OpenAI. (2026). GPT-5.5 model documentation. https://openai.com

[42] Anthropic. (2026). Claude Sonnet 4.6 model documentation. https://www.anthropic.com

[43] Google DeepMind. (2026). Gemini 3.5 Flash model documentation. https://deepmind.google

[44] xAI. (2026). Grok 4.6 Fast model documentation. https://x.ai

[45] DeepSeek. (2026). DeepSeek-V3 model documentation. https://www.deepseek.com

Appendices

Appendix A — C0 Grammar Specification

Source: `instruction.txt`. Lexical rules:

```
<Di>    -> 0 | ... | 9
  <DiS>   -> <Di> | <Di><DiS>
  <Le>    -> a | ... | z | A | ... | Z
  <DiLe>  -> <Le> | <Di>
  <DiLeS> -> <DiLe> | <DiLe><DiLeS>
  <Na>    -> <Le> | <Le><DiLeS>
  <C>     -> <DiS> | <DiS>u | null
  <CC>    -> ' ' | ... | '~'
  <BC>    -> true | false
  <id>    -> <Na> | <id>.<Na> | <id>[<E>] | <id>* | <id>&
```

Expression hierarchy: `<F>`, `<T>`, `<E>`, `<Atom>`; boolean: `<BF>`, `<BT>`, `<BE>`; statements: `<St>`, `<StS>`, `<rSt>`; program: `<prog>`, `<TyDS>`, `<VaDS>`, `<FuDS>`, `<FuD>`, `<body>`. Full productions are in `instruction.txt`.

Appendix B — Mermaid Derivation Tree Format Standard

Key rules (Output Format v2):

1. `graph TD`; plain edges; no styling or colour directives.
2. Non-terminals: `&lt;Sym&gt;`; terminals as separate leaf nodes.
3. Full lexical expansion; children in production order; root `<prog>`.
4. Exclude whitespace nodes and end-of-input markers.

Appendix C — Test Statements

From `accepted C0 lines.txt` (51 programs):

Tasks 1–5 (integer/unsigned constants):

```
01  int a; int main(){a = 2147483647; return 1}
  02  int a; int main(){a = -2147483647; return 1}
  03  uint a; int main(){a = 4294967295u; return 1}
  04  uint a; int main(){a = 0u; return 1}
  05  uint a; int main(){a = 123123u; return 1}
```

Tasks 45–48 (pointers):

```
45  typedef struct {int val} signed; typedef signed' ptr; ...
  46  typedef int' psigned; psigned ptr; ... ptr = b&; a = ptr'; ...
  47  typedef struct {int exp; int val} powst; typedef powst' ptr; ...
  48  typedef LEL' u; typedef struct {int content; u next} LEL; ...
```

Tasks 49–51 (out-of-grammar):

```
49  int main(){asm( addi 2 0 3 ); return 1}
  50  ... gpr(1) = a; q.pp.f = gpr(1); ...
  51  ... gpr(1) = val; asm( addi 1 1 1 ); ...
```

Appendix D — Reference Derivation Trees

Location: `generated_trees/`. Naming: `NN_descriptive_name.mmd` paired with `.c0` source. Example: Task 1 → `01_single_int_max_constant.mmd`.

Appendix E — Model Outputs and Results

Model	Answers folder	Results CSV
ChatGPT	`GPT_GPT-5.5_answers/`	`GPT_GPT-5.5_results.csv`
Gemini	`Gemini_3.5flash_answers/`	`Gemini_3.5flash_results.csv`
Claude	`Claude_Sonnet_4.6/`	`Claude_Sonnet_4.6_results.csv`
Grok	`Grok_4_6fast_answers/`	`Grok_4_6fast_results.csv`
DeepSeek	`DeepSeek-V3_answers/`	`DeepSeek-V3_results.csv`

Table 10: Data file locations for model outputs and evaluation results.

Each CSV contains 51 rows with metrics: legality, fullyLegal, structural, editDistance, border, edge, node, error counts, and normalization flags. Interactive charts: `c0_derivation_tree_thesis_dashboard.html`. Exported figures: `figures/`.